

x402-Shield · Status & Camadas de Proteção

Maio 2026 — pré-hackathon

2026-05-09

x402-Shield

HTTP 402 priority gate for Solana RPC operators

Camada de defesa, monetização e enforcement agêntico — pluga na frente de qualquer RPC Solana existente (Helius, Triton, Ankr, validador self-hosted). Open-source, drop-in, sem mudar o RPC upstream.

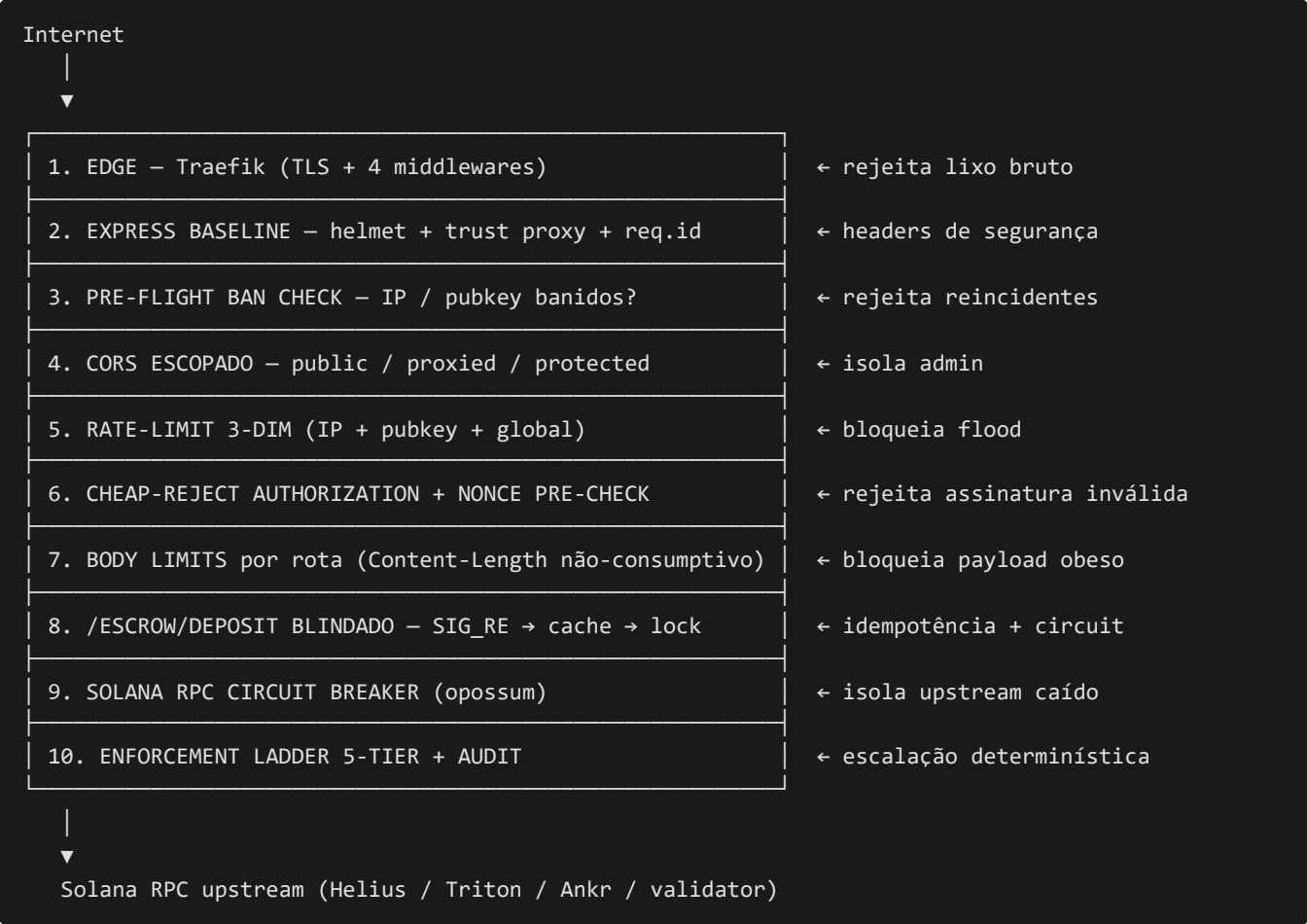
"It's not an error — it's an automated economic negotiation."

Status atual

Métrica	Valor
Branch	feat/anti-flood-defense-v2
Commits	70 (a partir de main em d9ed203)
Arquivos modificados	87
Linhas	+9 461 / -470
Pull Request	#1 — github.com/flavioparah/x402-priority-protocol/pull/1
Testes verdes	Phase 0-4 (~30 arquivos, 250+ asserções)
Cobertura defensiva	5 fases (Foundation, Edge, Core, Enforcement, Agent/Admin)
Documentação operacional	Runbook 487 linhas, FAQ defensivo, deploy guide

Defesa em profundidade — visão geral

Toda requisição passa por **10 camadas** antes de chegar ao Solana RPC upstream. Cada camada bloqueia uma classe específica de abuso:



Camada 1 — Edge Traefik

Middleware	Função	Limite
x402-ratelimit	Token-bucket por IP	30 req/s sustentado, burst 60
x402-inflight	Cap de conexões concorrentes	200 (cumulativo devnet+mainnet)
x402-bodylimit	Rejeita POST > limite	64 KB
x402-headers	HSTS 1y + nosniff + referrer-policy + remove Server / X-Powered-By	—

Característica: roda antes do Node. Lixo bruto nunca toca CPU do Shield.

Smokes: 4 scripts shell em tools/edge-smoke/ com --dry-run .

Camada 2 — Express baseline

Componente	Defesa
<code>helmet</code>	CSP, HSTS 1 ano, X-Frame-Options DENY, no-sniff, referrer-policy
<code>app.set('trust proxy', 1)</code>	IP real do cliente (não da VPS) — confia em 1 hop apenas
<code>app.disable('etag')</code> + <code>disable('x-powered-by')</code>	Sem cache stale, sem fingerprint do framework
<code>app.set('query parser', 'simple')</code>	Rejeita <code>?foo[bar]=baz</code> (prototype pollution)
Middleware de correlation ID	<code>req.id</code> 8-hex, header <code>X-Request-ID</code> na resposta. Servidor sempre gera; ignora valor do cliente (anti log-injection)

Camada 3 — Pre-flight ban check

```
req.ip          → checkBan(`ip:${ip}`)          → tier 2/3/4? → 429/403 + Retry-After  
X-x402-Agent-Pk → checkBan(`pk:${pubkey}`)          → tier 2/3/4? → 429/403 + Retry-After
```

Roda **antes** de qualquer outra defesa. Reincidentes nunca custam CPU adicional.

Headers retornados: `X-x402-Reason` (vocabulário fechado), `X-x402-Tier`, `X-x402-Until`, `X-x402-Trust-Impact`.

Camada 4 — CORS escopado

Categoria	Rotas	Access-Control-Allow-Origin
Public read-only	<code>/info</code> , <code>/health</code> , <code>/stats/*</code> , <code>/reputation/*</code> , <code>/escrow/balance/*</code> , <code>/agent/code-of-conduct</code>	<code>*</code>
Proxied	<code>/rpc</code>	<code>*</code> (compat SDK)
Protected	<code>/escrow/deposit*</code> , <code>/admin/*</code>	só Origin na allowlist (<code>PROTECTED_ORIGIN_ALLOWLIST</code>)
Default	qualquer outra	sem ACAO (server-to-server passa)

Camada 5 — Rate-limit 3-dimensional

Cada `/rpc` consome 3 buckets independentes em ordem (Redis ZSET sliding-window via Lua atomic):

1. global	rl:global	5000 req/min	← total do operador
2. ip	rl:rpc:ip:{ip}	100 req/min	← por IP
3. pubkey	rl:rpc:pk:{pubkey}	200 req/min	← por agente autenticado
4. paid	rl:rpc:paid:{pubkey}	200×N req/min	← lane premium aditiva

Paid lane = aditiva, NÃO bypass. Pubkey que paga consome IP + pubkey + global + paid (mesmo se for whale).

Trust multiplier:

Trust Score	Bucket paid ×	Característica
0–20	1×	Novo / suspeito
21–50	2×	Estabelecido
51–80	5×	Confiável (imune a tier-4 auto)
81–100	10×	Premium (ban requer co-evidência de fraude)

Camada 6 — Cheap-reject + nonce pre-check

Defesa contra flood de `Authorization: x402 <lixo>`:

```
Etapa 1: preflightAuth(header)
- regex de comprimento (sig 87-88, pubkey 32-44, msg 50-500 base58)
- retorna reason em ~1µs

Etapa 2: noncePreCheck(parts, store)
- bs58.decode(parts[2]) bounded em 1024 bytes
- JSON.parse → lê SOMENTE payload.nonce
- lookup no store; falha → reason fechada

Etapa 3 (só se 1+2 passarem): nacl.sign.detached.verify
```

Garantia provada por teste: 1000 headers garbage → 0 chamadas a `nacl.verify` e `bs58.decode`. Verificado por spy em `test/cheap-reject.test.js`.

Invariante de segurança: `payload.pubkey`, `payload.amount`, `payload.destination` nunca são lidos antes de `nacl.verify` autenticar a mensagem (verificado por Proxy trace em `test/nonce-precheck-bounded.test.js`).

Camada 7 — Body limits por rota

Rota	Mecanismo	Limite
<code>/rpc</code>	Content-Length header inspection (não consome stream)	32 KB
<code>/escrow/deposit*</code>	<code>express.json({ limit: '1kb' })</code>	1 KB
<code>/admin/*</code>	<code>captureRawBody</code> middleware	4 KB

Crítico em `/rpc` : o middleware **nunca consome** o body. `http-proxy-middleware` continua bombeando bytes intactos pro Solana. Verificado por mock upstream que checa byte-equality.

Camada 8 — /escrow/deposit blindado

5 portões antes de ligar pro Solana:

```
1. SIG_RE.test(sig)           → 400 invalid_signature_format
2. isDepositKnownBad(sig)     → 400 cached_negative (60s TTL)
3. claimPendingDeposit(sig, 15s NX) → 409 deposit_in_progress + Retry-After
4. fireSolanaCircuit(sig)     → 503 circuit_open + Retry-After:30
5. validation result.ok === true → markDepositKnownBad(sig, 60s) se falhar
finally: clearPendingDeposit(sig)
```

Resultado: N requisições concorrentes com a **mesma sig** → Solana é chamado **exatamente uma vez**.

Camada 9 — Solana RPC circuit breaker

opossum config:

Parâmetro	Valor
Threshold	≥ 50% erros sobre janela de 30s
Reset timeout	30s (OPEN → HALF_OPEN)
Volume threshold	5 requisições mínimas
Per-call timeout	15s

Estados: `CLOSED` (normal) → `OPEN` (Solana caído) → `HALF_OPEN` (tentativa) → `CLOSED`.

Quando OPEN, todas as chamadas `getParsedTransaction` retornam `503 + Retry-After: 30` instantaneamente (sem custo de I/O).

Camada 10 — Enforcement ladder 5-tier

Determinístico. Mesma entrada → mesma resposta. Sem mágica.

Tier	Nome	Trigger	Duração	Ação
0	Warning	80% do bucket atingido	—	Header <code>X-x402-Trust-Impact: warn</code> , request passa
1	Throttle	Bucket excedido	—	429 + <code>Retry-After</code>
2	Soft-ban	3 throttles em 5 min OU 10 invalid-sig em 60s	5 min	403
3	Hard-ban	3 soft-bans em 24h OU sinal de fraude + throttle	1 h	403
4	Permanent	3 hard-bans em 7d	sem TTL	403

Permanent é DESLIGADO por default em mainnet (`ENFORCEMENT_TIER_MAX=3`). Tier 4 só via:

- 1. Operador chamar `/admin/ban` com `target_type: pubkey, tier: 4, manual_promotion: true`
- 2. Pubkey **fora** da janela de whitelist (30 dias / 10 pagamentos)
- 3. Score < 51 (acima desse threshold é imune a tier-4 auto)
- 4. Score < 81 ou fraud signal corroborado (≥81 exige co-evidência)

Logado integralmente em `audit:admin` + push em `audit:abuse:history` por pubkey/IP.

Endpoints públicos

Path	Método	Função	Rate-limit
/health	GET	Status JSON	120/min/IP
/info	GET	Operador metadata, network, pricing	120/min/IP
/stats/recent	GET	20 últimos pagamentos	60/min/IP
/stats/qos	GET	Métricas QoS	60/min/IP
/stats/leaderboard	GET	Top trust-scores	60/min/IP
/reputation/:pubkey	GET	Trust-Score + risco + atestações	30/min/IP
/escrow/balance/:pubkey	GET	Saldo em μ L	60/min/IP
/escrow/deposit	POST	Verifica tx on-chain → credita	5/min/IP
/escrow/deposit-trusted	POST	(devnet only) Credita sem on-chain	5/min/IP
/rpc	POST	Proxy x402 challenge → pagamento → forward	IP+pubkey+global+paid
/agent/code-of-conduct	GET	Documento JSON imutável v1.0	120/min/IP
/agent/status?pubkey=...	GET	Snapshot do agente (trust, ban, throttles)	10/min/IP, cache 10s
/metrics	GET	Prometheus exposition	10/min/IP

Endpoints agênticos `/admin/*`

Auth: HMAC-SHA256 canonical string + Origin allowlist + mass-ban guard.

```
canonical = method\npath\nquery_sorted\ntimestamp\nkey_id\nsha256_hex(body)
```

Path	Método	Função
<code>/admin/abuse-log</code>	GET	Audit log paginado (since, limit, type)
<code>/admin/agent/:pubkey</code>	GET	Diagnóstico completo do agente
<code>/admin/ban</code>	POST	Tier 2/3/4 (tier-4 requer <code>manual_promotion: true</code>)
<code>/admin/unban</code>	POST	Remove ban + permanente
<code>/admin/config</code>	GET	Snapshot de config efetiva
<code>/admin/config</code>	POST	Hot-reload via whitelist (RATE_, RPC_LOAD_, etc.)

Mass-ban guard: 10 bans/min por keyId, 50 bans/h global. Excedeu → 429 + audit alert.

Sem `ADMIN_KEYS_JSON` no env: `/admin/*` retorna 503 com `X-Admin-Status: not_configured`.

Container hardening

```
user: "1000:1000"           # uid 1000 (node), nunca root
read_only: true             # rootfs imutável
tmpfs: [/tmp]               # apenas /tmp gravável
cap_drop: [ALL]             # zero capabilities Linux
security_opt: [no-new-privileges:true]
ulimits: { nofile: 65535 }
stop_grace_period: 30s      # SIGTERM → drain QoS → exit 0
```

Dockerfile : `npm ci --omit=dev` + `chown node:node` + `USER node` .

Observabilidade

Telemetria	Tecnologia	Acesso
Logs estruturados	pino async (file fd 1, sync:false)	stdout do container
Hot-path warnings	<code>samplerWarn</code> 1-em-50 após 100 eventos	sem amplificação sob flood
Audit log	streams <code>kind: audit</code> (deposits) e <code>kind: admin</code> (ações)	grep do stdout ou Loki
Métricas Prometheus	<code>prom-client</code>	<code>GET /metrics</code>
Correlation ID	<code>X-Request-ID</code> 8-hex	header em toda resposta

Métricas expostas em `/metrics` :

- `x402_requests_total{route, stage, outcome}` — counter de eventos por estágio do pipeline
- `x402_ratelimit_blocks_total{dimension, route}` — blocks por dimensão (ip / pubkey / global / paid)
- `x402_solana_circuit_state` — gauge 0=closed, 1=half_open, 2=open
- `x402_admin_actions_total{action, outcome}` — auditoria operacional
- `x402_store_healthy` — gauge 0/1 (ioredis flag)
- `x402_solana_rpc_duration_seconds` — histogram 10-bucket
- `x402_abuse_events_total{reason}` — eventos da ladder
- Default Node.js metrics (heap, GC, event-loop lag)

Boot guards

3 verificações antes de aceitar tráfego:

Guard	Condição	Ação
A — Trusted+Mainnet	<code>ESCROW_TRUST_DEPOSITS=1</code> AND mainnet upstream	<code>process.exit(1)</code>
B — Redis Required	<code>REDIS_REQUIRED=true</code> (default em mainnet) AND Redis indisponível por 30s	<code>process.exit(1)</code>
C — Admin Not Configured	<code>ADMIN_KEYS_JSON</code> ausente	Mount stub 503 com <code>X-Admin-Status: not_configured</code>

Warning não-bloqueante: `RPC_LOAD_FORCE` em mainnet → log.warn destacado (modo demo).

Graceful shutdown

```
SIGTERM ou SIGINT
↓
shuttingDown = true
/health passa a retornar 503 status=shutting_down
↓
server.close() – refusa novas conexões
↓
poll a cada 200ms – espera qosInFlight=0 && qosQueue=0
deadline 25s
↓
store.close() – flush Redis
↓
setTimeout(100ms) → process.exit(0)
```

Compose alinha com `stop_grace_period: 30s` (5s de margem).

Testes — cobertura

Fase	Arquivos	Asserções aprovadas
Phase 0 (foundation, store, boot)	11	70+
Phase 2 (preflight, rate-limit, body, deposit, circuit, CORS)	9	80+
Phase 3 (enforcement ladder)	10	84
Phase 4 (admin, agent, metrics, config)	8	130+
Total novo	38	365+
Pré-existentes (smoke + atomic-consume + cooperative-qos)	3	21 (todos verdes após fix)

Todos rodam via `npm test` ou `npm run test:phase{0..4}` .

Pacote agêntico — para AI Agents

`/agent/code-of-conduct` retorna documento JSON imutável v1.0 contendo:

- **Rate budgets** (per_ip, per_pubkey, global, paid_pubkey base)
- **Backoff protocol** (429 → exponential backoff, 402 → pay challenge, 503 → wait 30s)
- **Identity rules** (pubkey rotation policy, nonce single-use)
- **Deposit rules** (signature validation, on-chain confirmation)
- **Enforcement tiers** com triggers e durações
- **Trust multipliers** lookup
- **Operator obligations** (audit retention, key rotation)

Versão semver. Bumps de major = breaking SDK contract change.

Decisões para hackathon

Networks ativas:

URL	Tipo	Custo SOL	Para quê
api.rpcpriority.com	Mainnet	Real	Demo “real deal” — pagamento on-chain
devnet.rpcpriority.com	Devnet	Faucet (grátis)	Demo de mecânica + Trust-Score progression
demo.rpcpriority.com	Demo	—	Aposentado (incompatível com Phase 0 Boot Guard A)

Modo demo: RPC_LOAD_FORCE=0.9 em ambos compose → toda chamada /rpc força challenge 402 (mostra o protocolo sem precisar de carga real).

Riscos remanescentes (todos baixos):

Risco	Severidade	Mitigação
Júri tenta /escrow/deposit mainnet sem SOL	Baixa	Mensagem transaction not found é clara; demonstre em devnet primeiro
6+ deposits rápidos → 429	Baixa	Cenário improvável; uso normal não dispara
Flood acidental → IP soft-banned 5min	Baixa	TTL 5 min; reseta após restart container

Próximos passos (24h até hackathon)

```
# 1. Pull no kvm4
ssh kvm4
cd /root/x402
git pull origin feat/anti-flood-defense-v2

# 2. Subir devnet + mainnet (NÃO subir docker-compose.yml raiz - incompatível)
docker compose -f docker-compose.devnet.yml up -d --build
docker compose -f docker-compose.mainnet.yml up -d --build

# 3. Smoke pós-deploy
curl https://devnet.rpcpriority.com/health
curl https://api.rpcpriority.com/info
bash tools/edge-smoke/test-security-headers.sh
bash tools/edge-smoke/test-bodylimit.sh

# 4. Validar fluxo SDK em devnet (do Laptop)
SHIELD_URL=https://devnet.rpcpriority.com npm run demo:trust
```

Pitch — pontos memoráveis para o júri

1. **Drop-in middleware** — pluga na frente de qualquer Solana RPC sem mudar o upstream
 2. **DDoS vira receita** — ataques pagam o operador (alinhamento de incentivos)
 3. **Defesa em 10 camadas** — não é uma feature, é um sistema completo
 4. **Atomic atomic-atomic** — Redis Lua scripts garantem zero double-spend, zero race
 5. **Public good open-source** — qualquer operador (Helius, Triton, validador) pode auto-hospedar
 6. **Agentic-first** — Code of Conduct versionado, /admin/* HMAC, /metrics Prometheus, sem painel HTML
 7. **Trust-Score portátil** — reputação cross-operator que cresce com pagamentos
 8. **Cheap-reject de assinatura** — flood de 1000 headers garbage = 0 nacl.verify (provado por teste)
 9. **Container hardened** — uid 1000 + read_only + cap_drop ALL + stop_grace_period 30s
 10. **365+ testes verdes** + runbook operacional de 487 linhas
-

Gerado em 2026-05-09. Branch `feat/anti-flood-defense-v2` @ commit `da8bf1a` . Pull Request: github.com/flavioparah/x402-priority-protocol/pull/1